



UNIVERSIDAD AUTÓNOMA DE SAN LUIS POTOSÍ
FACULTAD DE ESTUDIOS PROFESIONALES ZONA MEDIA

Práctica 12: Control de acceso mediante contraseña

Nombre del alumno: Sergio Adolfo Juárez Mendoza

Clave: 369376

Profesor: Ing. Jesús Padrón

Fecha de entrega: 21 de abril de 2025

Introducción

Para la practica numero doce se nos dio a la tarea de hacer un control de acceso mediante una contraseña ya que con lo antes ya hecho ya teniamos los conocimientos suficientes para poder hacerlo de manera correcta

Objetivo

poder mediante el monitor serial y el el teclado matricial pedir la contraseña asi mismo poder ingresarla con el teclado matricial

1 Material y Recursos Necesarios

1.1 Hardware

- Raspberry Pi Pico W
- teclado matricial
- Diodos emisores de luz(LED) rojo y verde
- Protoboard
- Cables de conexión calibre 22 AWG

1.2 Software y Librerías

- Visual Studio Code con soporte para Raspberry Pi Pico
- Librerías:
 - `pico/stdlib.h`
 - `hardware/adc.h`

Desarrollo

creacion del codigo

```

1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "hardware/gpio.h"
4
5 #define ROWS 4
6 #define COLS 4
7 #define LED_VERDE 1
8 #define LED_ROJO 2
9
10 const uint8_t row_pins[ROWS] = {6, 7, 8, 9};
11 const uint8_t col_pins[COLS] = {10, 11, 12, 13};
12 char keys[ROWS][COLS] = {
13     {'1', '2', '3', 'A'},
14     {'4', '5', '6', 'B'},
15     {'7', '8', '9', 'C'},
16     {'*', '0', '#', 'D'}
17 };
18 const char PASSWORD[] = "369376";
19 char input[7];
20 int index = 0;
21 int incorrect_attempts = 0;
22
23 void setup() {
24     stdio_init_all();
25     printf("Bienvenido, ingrese la contrase a:\n");
26     for (int i = 0; i < ROWS; i++) {
27         gpio_init(row_pins[i]);
28         gpio_set_dir(row_pins[i], GPIO_OUT);
29     }
30     for (int i = 0; i < COLS; i++) {
31         gpio_init(col_pins[i]);
32         gpio_set_dir(col_pins[i], GPIO_IN);
33         gpio_pull_up(col_pins[i]);
34     }
35     gpio_init(LED_VERDE);
36     gpio_set_dir(LED_VERDE, GPIO_OUT);
37     gpio_init(LED_ROJO);
38     gpio_set_dir(LED_ROJO, GPIO_OUT);
39 }
40
41 char get_key() {
42     for (int row = 0; row < ROWS; row++) {
43         gpio_put(row_pins[row], 0);
44         for (int col = 0; col < COLS; col++) {
45             if (gpio_get(col_pins[col]) == 0) {
46                 while (gpio_get(col_pins[col]) == 0);
47                 gpio_put(row_pins[row], 1);
48                 return keys[row][col];
49             }
50         }
51         gpio_put(row_pins[row], 1);
52     }
53     return 0;
54 }

```

```

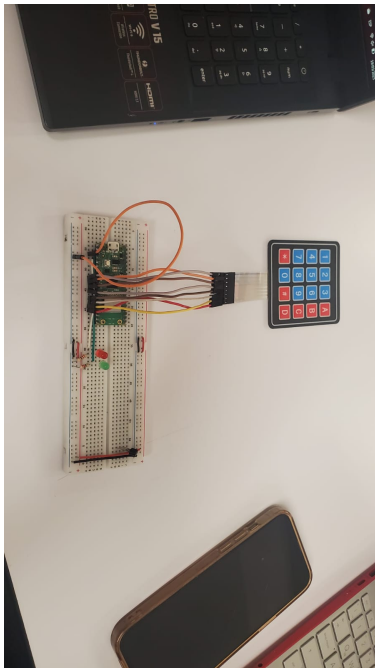
55
56 void lock_device() {
57     printf("\nDispositivo bloqueado. Espere 15 segundos.\n");
58     for (int i = 0; i < 15; i++) {
59         gpio_put(LED_ROJO, 1);
60         sleep_ms(500);
61         gpio_put(LED_ROJO, 0);
62         sleep_ms(500);
63     }
64     incorrect_attempts = 0;
65     printf("\nIngrese la contrase a nuevamente:\n");
66 }
67
68 void loop() {
69     char key = get_key();
70     if (key) {
71         printf("%c", key);
72         if (key == '#') {
73             input[index] = '\0';
74             if (strcmp(input, PASSWORD) == 0) {
75                 printf("\nContrase a correcta!\n");
76                 gpio_put(LED_VERDE, 1);
77                 gpio_put(LED_ROJO, 0);
78                 sleep_ms(5000);
79                 gpio_put(LED_VERDE, 0);
80                 incorrect_attempts = 0;
81             } else {
82                 printf("\nContrase a incorrecta!\n");
83                 gpio_put(LED_VERDE, 0);
84                 gpio_put(LED_ROJO, 1);
85                 sleep_ms(5000);
86                 gpio_put(LED_ROJO, 0);
87                 incorrect_attempts++;
88             }
89             index = 0;
90             if (incorrect_attempts >= 3) {
91                 lock_device();
92             } else {
93                 printf("\nIngrese la contrase a nuevamente:\n");
94             }
95         } else if (index < 6) {
96             input[index++] = key;
97         }
98     }
99     sleep_ms(200);
100 }
101
102 int main() {
103     setup();
104     while (1) {
105         loop();
106     }

```

Resumen del funcionamiento

Este programa implementa una **cerradura digital** utilizando un **teclado matricial 4x4** y dos LEDs conectados a una **Raspberry Pi Pico**. El objetivo es que el usuario ingrese una contraseña y, según su validez, se encienda un LED verde (acceso correcto) o uno rojo (acceso denegado). Después de tres intentos fallidos, el sistema se bloquea por 15 segundos.

Componentes principales



- **Teclado 4x4:** Se conecta a los pines GPIO de la Raspberry. Se escanean las filas y columnas para detectar teclas presionadas.
- **LED Verde (GPIO 1):** Se enciende si la contraseña es correcta.
- **LED Rojo (GPIO 2):** Se enciende si la contraseña es incorrecta o durante el bloqueo.
- **Contraseña:** El usuario debe ingresar la clave 369376 y confirmar con la tecla #.

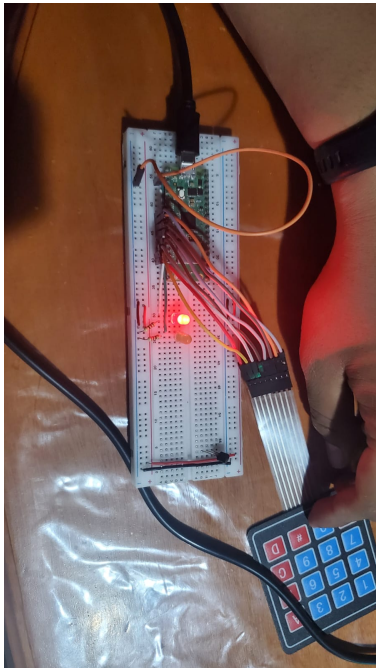
Flujo del programa

1. **Inicialización (setup):** Se configuran los GPIO de filas como salida, columnas como entrada con resistencias `pull-up`, y se inicializan los LEDs.
2. **Lectura de teclas (get_key):** Se pone a nivel bajo una fila y se verifica si alguna columna está en bajo (tecla presionada). Si se detecta una, se devuelve el carácter correspondiente.
3. **Verificación de contraseña (loop):**
 - Si se presiona una tecla, se imprime y se guarda en el arreglo de entrada.
 - Si se presiona `#`, se compara lo ingresado con la contraseña.
 - Si es correcta: se enciende el LED verde durante 5 segundos.
 - Si es incorrecta: se enciende el LED rojo durante 5 segundos y se incrementa el contador de intentos fallidos.
 - Si se alcanzan 3 intentos fallidos, se llama a la función de bloqueo.
4. **Bloqueo del sistema (lock_device):** El LED rojo parpadea durante 15 segundos. Luego se reinician los intentos fallidos y el usuario puede volver a intentar.

Consideraciones adicionales

- El arreglo de entrada debe ser limpiado tras cada intento para evitar errores.
- El `sleep_ms(200)` al final de `loop` ayuda a reducir rebotes del teclado y lecturas múltiples.
- El sistema es extensible a más funciones como cambio de clave o indicadores sonoros.

funcionamiento



Conclusion

El desarrollo de esta cerradura digital con teclado matricial 4x4 y control por medio de una Raspberry Pi Pico demuestra cómo es posible implementar sistemas de seguridad básicos combinando electrónica digital y programación en C. El proyecto permite reforzar conocimientos sobre el manejo de entradas y salidas GPIO, escaneo de teclados, control de LEDs, estructuras de control y temporización.

Además, se introducen conceptos fundamentales en la protección de acceso, como la verificación de contraseñas, el manejo de errores y la implementación de mecanismos de bloqueo tras múltiples intentos fallidos. Este sistema, aunque sencillo, puede servir como base para proyectos más complejos en domótica, control de acceso o sistemas embebidos de seguridad.